Evidence-Guided Small-Part Sorting with a FANUC Digital Twin

Full project narrative • Rockwell Automation + NVIDIA Innovation Lab • Grasping and Sorting

Project overview

We built a modular FANUC ER-4iA and Isaac Sim pipeline that observes colored cubes and M6x30 screws, estimates their poses, generates feasible grasp candidates, sorts the objects into predetermined destinations, verifies the final outcome, and records failures. Red cubes and screws are assigned to the red destination; the other colored cubes are assigned to their matching destinations.

Our central engineering question was not merely whether artificial intelligence could be added to the system. We wanted to measure whether pretrained AI actually improved object perception or grasp selection compared with a reliable deterministic baseline, then retain the method supported by evidence.

The manufacturing problem

Small-part sorting is sensitive to perception error, occlusion, camera calibration, part dimensions, clutter, and grasp choice. Parts may arrive at different positions and rotations, overlap, or partially block one another. A fixed pick routine may miss a part, approach it from a blocked direction, or acquire it without retaining it throughout transport.

These failures have manufacturing consequences. A dropped or misplaced part requires a retry or operator intervention, consumes cycle time, reduces throughput, and may interrupt downstream work. This is why we prioritized **correct placement and consistency** over nominal speed. A fast attempt is not valuable if it causes a failed pickup and a longer recovery cycle.

Our hypothesis

We asked: **Can AI be used not only to detect objects, but also to improve grasp selection?**

The proposed idea was to generate several predefined, physically feasible grasps for each detected object and use an AI model at runtime to select the grasp least likely to fail. In this project, a grasp succeeds only if the robot:

1. acquires the intended object;
2. retains it throughout the entire relocation motion; and
3. releases it only at the intended destination.

The AI was never intended to make an unsafe grasp safe. Deterministic geometry remained responsible for candidate generation and for rejecting candidates that violated reachability, finger fit, or known clearance constraints. AI was evaluated only as a perception aid or as a ranker among candidates that had already passed those checks.

Scope and assumptions

The prototype was developed under the following assumptions:

- A small wall or other bounded working area prevents most objects from leaving the workspace.
- Objects are sufficiently durable that an occasional drop does not shatter or permanently deform them.
- Objects are relatively non-bouncy and are not expected to rebound over the workspace boundary onto the floor.
- The supported part families, dimensions, symmetries, and sorting rules are known in advance.
- Sorting destinations are predetermined.
- A destination does not become permanently full; it is either large enough for the demonstrated workload or emptied by a separate process.
- Cubes are upright, and screws are isolated and lying on the table for the validated pilot.
- The camera has enough visibility to produce a usable pose estimate.

Arbitrary piles, deformable parts, highly reflective finishes, filled destinations, and unrestricted moving arrivals were not validated. These assumptions define the manufacturing problem we attempted to solve; they are not claims that the system

works immediately for every possible part or factory environment.

System approach

The pipeline converts rendered camera images into structured object observations containing class, position, orientation, confidence, and provenance. It then:

1. converts image observations into robot-base coordinates;
2. generates several candidate grasps;
3. rejects candidates that violate geometric or robot constraints;
4. selects a remaining grasp;
5. executes the motion in Isaac Sim;
6. verifies whether the object ended inside the correct destination; and
7. logs the decision, execution, success, and failure information.

Simulator ground truth was reserved for evaluation and was not used as a hidden runtime shortcut.

We evaluated two pretrained AI components:

- **YOLO11n for perception.** It was compact, fast, and the strongest neural detector in our limited cube-focused evaluation. It was integrated as an assistant to calibrated geometric pose estimation rather than as a source of robot coordinates by itself.
- **NVIDIA Cosmos3-Nano for grasp ranking.** We selected it as a pretrained physical-reasoning model that could receive structured, already-feasible candidates without requiring us to collect enough data to train a large reasoning model from scratch. Cosmos was not trained by our team, and its confidence was not treated as a calibrated probability of complete grasp success.

These selections were hypotheses to test, not predetermined winners.

Experimental design

We compared four combinations:

- **DD:** deterministic vision and deterministic grasp ranking;
- **DA:** deterministic vision and guarded Cosmos grasp ranking;
- **YD:** YOLO-assisted vision and deterministic grasp ranking; and
- **YA:** YOLO-assisted vision and guarded Cosmos grasp ranking.

Ten preregistered randomized Isaac Sim scenes were used, one at each object count from one through ten. The same scene seeds were replayed under all four policies. This produced 40 complete end-to-end runs and 55 object opportunities per policy. The simulation varied object mixtures and poses, lighting, materials, camera and sensor effects, and physics parameters. All expected reports completed, with no missing reports or episode-level infrastructure failures.

Measured results

Policy	Correctly sorted objects	Complete scenes	Correctly sorted per simulated minute
DD	**45/55 (81.8%)**	**6/10**	**4.97**
DA	38/55 (69.1%)	3/10	3.76
YD	14/55 (25.5%)	1/10	4.28
YA	9/55 (16.4%)	1/10	3.09

The deterministic baseline was the strongest tested policy. DD exceeded DA by 15.2 percentage points in matched object completion, with a paired 95% confidence interval from 3.3 to 29.9 percentage points. Cosmos changed the deterministic grasp choice 21 times, but only 9 of those changed choices succeeded.

The strict YOLO integration also performed poorly as a complete policy. It could reject an entire observation when one detected box disagreed with geometry. This means the low YD and YA end-to-end results do not prove that YOLO never recognized

individual cubes; they show that our all-or-nothing integration rule was too brittle for this task.

The evidence therefore did **not** justify deploying Cosmos or the strict YOLO gate as the default. We retained deterministic vision and deterministic grasp selection for the demonstrated pipeline. This is not evidence that learning can never help. It means these particular pretrained models, prompts, and integration rules did not outperform the simpler baseline in our matched pilot.

Physical equipment and validation limitation

We used the organizer-provided FANUC ER-4iA digital twin, robot and gripper geometry, screw CAD model, and Isaac Sim physics. We did **not** receive an opportunity to test the final pipeline on CSI hardware.

Consequently, the final system's physical camera calibration, controller timing, grip forces, collision clearances, and sim-to-real reliability remain unverified. The measurements above are simulation evidence, not proof of physical-robot performance. A responsible next hardware step would begin with camera and coordinate-frame validation, then hover-only motion, a single supervised pickup, grasp-retention verification, and only afterward complete sorting.

Scaling to new parts

The project does not claim to work out of the box for every possible part. Instead, it demonstrates a repeatable adaptation process that a factory team could apply to a new, bounded part family.

For a new part, the team would:

1. add its CAD model or measured dimensions;
2. document its coordinate axes, symmetries, and destination rule;
3. collect representative camera images across expected lighting, poses, and occlusions;
4. validate camera-to-robot pose error against the available grasp clearance;
5. define a small set of geometrically compatible grasp candidates;
6. collect grasp outcomes for those candidates;
7. train or adapt a task-specific detector or grasp-success ranker if needed; and
8. enable the learned component only after it beats the deterministic baseline on held-out scenes.

Demonstrating both cubes and geometrically different screws supports this scalability argument more strongly than changing only an object's color. A future experiment could repeat the same process from scratch for another part family and measure the adaptation effort and resulting performance.

Adapting to a new gripper

The mentor guidance correctly emphasized that we did not need to manufacture new fingers merely to complete the project. More important was explaining how the method would adapt when a factory uses a different gripper.

A gripper change requires updating the finger geometry, jaw stroke, tool-center point, collision envelopes, grasp-width limits, and candidate-generation rules. The perception and evaluation interfaces can remain the same. This modularity means a new gripper should require reconfiguration and revalidation rather than a complete pipeline rewrite. Any learned grasp ranker would also need representative outcomes collected with the new hardware because success probabilities do not automatically transfer between grippers.

Project autopsy: what limited us

Several process problems reduced what we could validate:

- **Late start.** Midterms delayed meaningful project work, leaving roughly two weeks for implementation and integration.
- **Inefficient use of the lead developer's ChatGPT subscription.** Non-lead contributors were not always given code-complete, bounded starting points. They then had to spend limited AI usage adding foundational code and sometimes reached usage limits before integration was complete.
- **Tasks were delegated too late.** Teammates received perception and grasping assignments after too much of the schedule had already elapsed, leaving insufficient time for iteration and integration. AI usage limits intensified this problem.

- **Technical setup problems consumed meetings.** Large portions of shared meeting time were spent resolving environments, credentials, dependencies, and machine-specific issues instead of integrating and testing the system.
- **Physical validation was not completed.** We never obtained an opportunity to test the final pipeline on the real CSI robot, leaving the most consequential sim-to-real assumptions unresolved.

These are process findings, not excuses for the measured limitations. They explain which changes would most improve a future attempt.

How we would improve the next project

- Begin technical work immediately after kickoff, even during a short planning phase, by reproducing the starter system and scheduling physical access.
- Assign independent workstreams and interfaces early. Each teammate should receive a concrete first deliverable, an owned folder and branch, a small passing test, and a definition of done.
- Give contributors code-complete starting points for shared infrastructure. Limited AI subscriptions should be used for bounded team-specific work rather than reconstructing missing integration foundations.
- Budget AI usage deliberately. The lead should prepare reusable setup instructions, fixtures, schemas, and examples before delegating work, while teammates preserve progress through frequent commits and written decision logs.
- Establish a preliminary troubleshooting period before each meeting. The team lead can resolve environment and access issues before the main collaboration window so meeting time is used for integration and evaluation.
- Validate the smallest end-to-end path first: one camera-observed object, one feasible grasp, one destination, verified containment, and a trustworthy log.
- Schedule hardware access early and freeze a tested version before the session. Simulation development should continue, but it should not substitute for calibration and supervised physical evidence.
- Treat every AI component as an experiment against a simpler baseline. Keep it only when the measured benefit justifies the added integration and debugging burden.

We will participate in another hackathon in mid-November and use it as an immediate opportunity to apply these lessons. We will begin work at kickoff, delegate bounded workstreams earlier, give contributors code-complete integration points, resolve setup problems before team meetings, and schedule physical validation as early as the event permits.

Conclusion

Our initial idea was to improve grasp selection with pretrained AI. The more important result was learning to test that idea honestly. In the matched Isaac Sim pilot, deterministic perception and deterministic grasp selection produced the highest object-completion rate, the most complete scenes, and the highest correctly sorted throughput. We therefore retained the consistent baseline instead of forcing an AI component into the deployed path.

The modular interfaces still provide a path to task-specific learning. A future factory team can add a defined part family, representative images, compatible grasp candidates, and measured outcomes, then train or adapt a model and compare it with the same baseline. The system's value is not a claim of universal autonomy; it is a reproducible process for turning a bounded sorting problem into a testable perception, grasping, execution, and evaluation pipeline.

Repository and reproducible benchmark results: <https://github.com/ALL-Applied-AI-Network/msoe-ai-club-rockwell-automation-nvidia-hackath-new-team-8-b27f0a8f50/tree/experiments/randomized-four-combo-v1>